

Página de criação *de artigos*

A redação está aberta e a Autora Teste tem o papel de autor. Agora ela ganha onde escrever: um editor com título, endereço, resumo, capa, tags e conteúdo em Markdown, rascunhos que só ela vê e um botão de publicar com checklist. No banco, a tabela de artigos e o armazenamento de capas nascem com as regras de quem pode o quê.

Tabela artigos

Rascunho × publicado

Só autores escrevem

Capas no Storage

Editor Markdown

Meus artigos

Formação Claude Code 2026 · Baseado em docs.lovable.dev (Connect to Supabase: Storage) e supabase.com/docs (Row Level Security), consultados em setembro de 2026 · Telas: reprodução ilustrativa

O que entra *hoje*

BANCO

- Tabela **artigos**
- Regras: só autor cria; rascunho só para o dono; publicado para todos

ARQUIVOS

- Bucket **capas** no Storage
- Só autores enviam, cada um na sua pasta
- Tamanho e tipo limitados

TELAS

- **Escrever**: editor completo
- **Publicar**: checklist antes
- **Meus artigos**: rascunhos e publicados

FICA PARA A AULA 03

A página pública do artigo, a lista na página inicial e o tempo de leitura. Hoje o foco é quem escreve.

Primeiro o banco e o Storage, depois a interface, em pedidos separados, como no módulo 28. Um pedido grande mistura as causas quando algo dá errado.

A tabela *artigos*

COLUNA	REGRA
<code>id</code>	Chave gerada automaticamente
<code>autor_id</code>	Dono; padrão <code>auth.uid()</code> , como no módulo 28
<code>titulo</code>	Obrigatório
<code>slug</code>	Endereço do artigo; único , só letras minúsculas, números e híffens
<code>resumo</code>	Até 200 caracteres
<code>conteudo</code>	Texto em Markdown
<code>capa_path</code>	Caminho da imagem no bucket <code>capas</code>
<code>tags</code>	Lista de até 5 tags (usadas na aula 10)
<code>status</code>	rascunho · publicado; padrão rascunho
<code>publicado_em</code>	Preenchido na primeira publicação
<code>criado_em</code> · <code>atualizado_em</code>	Automáticos

POR QUE UM SLUG

É o pedaço legível do endereço: `/artigos/como-organizo-minhas-ideias-com-ia` . Ajuda quem compartilha o link e os buscadores. Por isso precisa ser único e estável.

POR QUE PUBLICADO_EM SEPARADO

O artigo é criado como rascunho e publicado dias depois. A data que o leitor vê é a da publicação, não a da criação.

Regras de acesso *dos artigos*

POLÍTICA	OPERAÇÃO	QUEM	CONDIÇÃO
Ler publicados	SELECT	Qualquer pessoa	<code>status = publicado</code>
Ler os próprios	SELECT	Autor logado	<code>autor_id</code> = id dele (inclui rascunhos)
Criar	INSERT	Autor logado	<code>tem_papel(autor)</code> e <code>autor_id</code> = id dele
Editar	UPDATE	Autor logado	Só os próprios; <i>with check</i> mantém o dono
Apagar	DELETE	Autor logado	Só os próprios

NA MIGRATION, CONFIRA

- ✓ RLS ativo em `artigos`
- ✓ INSERT usa a função `tem_papel` da aula 01
- ✓ UPDATE com *with check* (lição da aula 08 do módulo 28)
- ✓ Nenhuma escrita para visitantes ou leitores

O teste que importa: um leitor logado, sem papel de autor, tentando criar um artigo. A resposta tem de ser recusada **pelo banco**, mesmo que alguém force a requisição sem passar pela tela.

Capas *no Storage*

Imagens não vão dentro da tabela: ficam num **bucket** do Supabase Storage, e a tabela guarda só o caminho. Buckets têm permissões próprias, e o Lovable pede aprovação antes de criá-los ou alterá-los.

REGRA DO BUCKET CAPAS	POR QUÊ
Leitura pública	A capa aparece para qualquer visitante
Envio só por autores	Leitor não usa o blog como hospedagem de arquivos
Cada autor na sua pasta (<code>id-do-autor/arquivo</code>)	Um autor não substitui a capa de outro
Até 2 MB; JPG, PNG ou WebP	Página leve e sem arquivos estranhos

Público quer dizer público. Num bucket com leitura pública, quem tiver o endereço da imagem consegue abri-la, mesmo que o artigo ainda seja rascunho. Não envie como capa nada que não possa ser visto.

ONDE CONFERIR
No painel do Supabase, em **Storage**: o bucket `capas`, as pastas por autor e as permissões.

Dois pedidos: *banco e tela*

Pedido 1 (Plan mode) · banco e Storage:

Crie a tabela artigos (veja o Knowledge) com:
id, autor_id (padrão auth.uid()), titulo,
slug (único; minúsculas, números e hifens),
resumo (até 200), conteudo (Markdown),
capa_path, tags (até 5), status (rascunho |
publicado; padrão rascunho), publicado_em,
criado_em e atualizado_em.

RLS: todos leem publicados; o autor lê os
próprios; só quem tem papel autor cria
(tem_papel); editar e apagar só os próprios,
com with check no UPDATE.

Crie o bucket "capas": leitura pública;
envio só por autores, na pasta com o id
deles; até 2 MB; JPG, PNG ou WebP.
Mostre as migrations antes.

Pedido 2 (Plan mode) · interface:

Página /escrever (só autores):

- título grande; slug gerado do título,
editável, com aviso se já existir;
- resumo com contador (200);
- abas Escrever e Pré-visualizar para o
Markdown, sem permitir HTML cru;
- capa com envio e troca; tags (até 5);
- "Salvar rascunho" e "Publicar...", que
abre um checklist e só libera se tudo
estiver ok; mostre "Salvo às hh:mm".

Página /meus-artigos com abas Todos,
Rascunhos e Publicados.

Leitores que abrirem /escrever veem "Esta
área é só para autores".

O editor *de artigos*



/escrever · reprodução ilustrativa com texto de exemplo

SLUG VISÍVEL

O endereço final aparece embaixo do título, editável.

RESUMO COM LIMITE

112 / 200: o contador evita o erro antes de salvar.

MARKDOWN

Títulos com `##`, listas e citações. A aba ao lado mostra o resultado.

STATUS SEMPRE VISÍVEL

"Rascunho · Salvo às 14:32" tira a dúvida "será que salvou?".

Publicar *com checklist*



Publicar artigo?

Depois de publicado, qualquer pessoa pode ler e compartilhar.

- ✓ Título preenchido
- ✓ Resumo com até 200 caracteres
- ✓ Capa enviada
- ✓ Pelo menos uma tag
- ✗ Conteúdo com pelo menos 300 caracteres (faltam 42)

[Voltar](#) [Publicar agora](#)

Checklist antes de publicar · reprodução ilustrativa

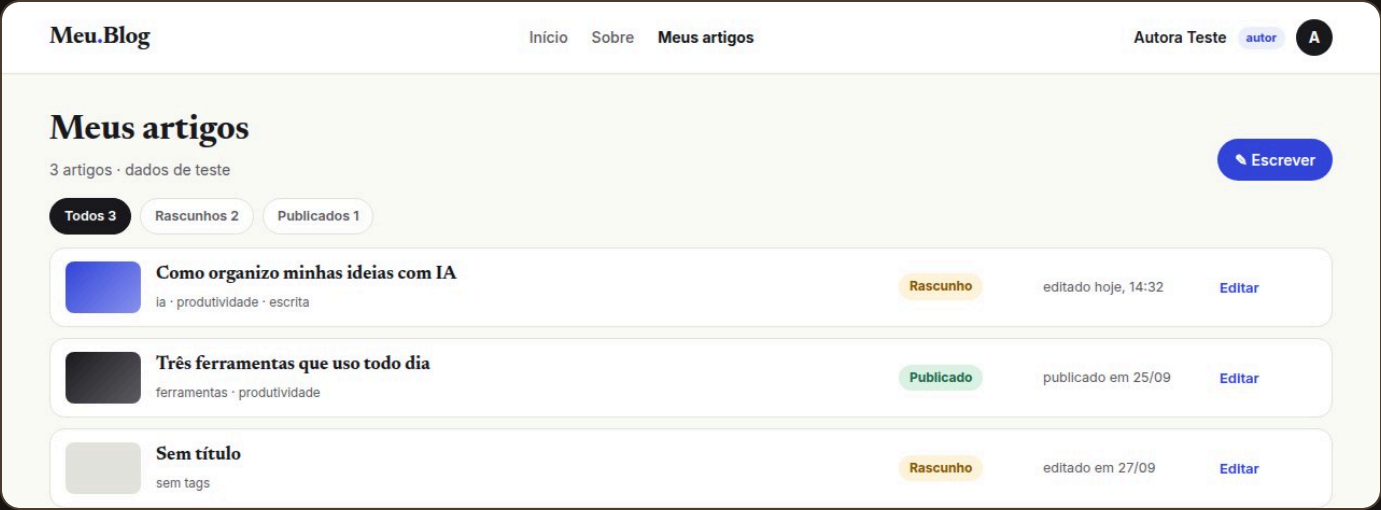
- ✓ Diz a consequência: **qualquer pessoa pode ler**
- ✓ Mostra o que falta ("faltam 42") em vez de só bloquear
- ✓ O botão fica desabilitado até tudo estar ok
- ✓ Na primeira publicação, `publicado_em` é preenchido

EDITAR UM ARTIGO PUBLICADO

As mudanças aparecem para todos ao salvar. Mudar o **slug** de um artigo já publicado quebra os links compartilhados; o editor deve avisar antes.

Despublicar (voltar para rascunho) tira o artigo da página pública, mas não apaga. É o equivalente ao "cancelar sem apagar" da Café Samambaia.

Meus *artigos*



/meus-artigos · reprodução ilustrativa com dados de teste

SÓ OS DA AUTORA

A lista vem filtrada pelo RLS: nenhum filtro "à mão" no código.

STATUS COM COR E TEXTO

Rascunho em amarelo, Publicado em verde, sempre escritos.

"SEM TÍTULO"

Rascunho salvo sem título aparece assim, nunca como linha vazia.

Markdown *sem HTML cru*

Markdown é seguro porque é limitado: títulos, listas, negrito, links. O risco aparece quando o texto pode trazer **HTML** ou **scripts** junto e isso é exibido para os leitores.

A REGRA

- ✓ O conteúdo é mostrado como Markdown, **sem HTML cru**
- ✓ A pré-visualização usa o **mesmo** componente que a página pública da aula 03
- ✓ Links externos abrem em nova aba, sem dar acesso à página de origem

Teste (Chat mode):

Se um autor escrever uma tag `<script>` no conteúdo, o que acontece na pré-visualização e na página pública? O HTML é ignorado ou executado? Não altere nada.

Hoje só a Autora Teste escreve, mas o blog pode ganhar outros autores. Tratar o conteúdo de qualquer autor como texto, e nunca como código, protege os leitores.

Três contas, *cinco provas*

1 **Autora:** cria um rascunho, envia uma capa, salva. Aparece em Meus artigos.

2 **Visitante** (janela anônima): o rascunho não é encontrado em lugar nenhum.

3 **Leitor** logado: abre /escrever e vê "só para autores". Pergunte no Chat mode se ele consegue inserir em artigos pelo banco: não.

4 Tente um segundo artigo com o **mesmo slug**: aviso na tela, recusa no banco.

5 Tente enviar um PDF ou uma imagem de 5 MB como capa: recusado.

REGISTRO DOS TESTES **EXEMPLO**

Meu Blog • criação de artigos

✓ autora cria rascunho	ok
✓ capa na pasta da autora	ok
✓ visitante vê rascunho	não
✓ leitor cria artigo (banco)	recusado
✓ slug repetido	recusado
✓ capa PDF / 5 MB	recusada
✓ publicar com checklist	ok

bookmark: "artigos v1"

Publique um artigo de teste agora: a aula 03 precisa de pelo menos um publicado para mostrar a página do artigo.

Erros comuns *ao criar artigos*

Verificar autor só na tela. Esconder /escrever não impede um leitor de inserir artigos pela API. A política de INSERT usa tem_papel.

Exibir HTML cru. Um script no conteúdo rodaria no navegador de cada leitor. Markdown sem HTML.

Bucket sem regra de pasta. Um autor sobrescreve a capa de outro. Cada um na pasta com o próprio id.

Trocar o slug depois de publicar. Todo link compartilhado quebra. Avise antes de permitir.

Capas de 8 MB. A página inicial fica pesada no celular. Limite tamanho e formato.

Publicar sem querer. Sem checklist, um clique leva um texto pela metade ao público.

Teste o que *ficou*

1. Quem pode ler um rascunho?

A) Qualquer pessoa

B) Só o autor dele

C) Leitores logados

D) Ninguém

B. A política "ler os próprios" inclui rascunhos; a pública, só publicados.

2. Por que a capa fica no Storage, e não na tabela?

A) O banco não aceita texto

B) Arquivos ficam em buckets com permissões próprias; a tabela guarda o caminho

C) É mais barato

D) Por causa do slug

B. Bucket capas: leitura pública, envio só por autores, cada um na sua pasta.

3. O que impede um leitor de criar artigo pela API?

A) O botão escondido

B) A política de INSERT com `tem_papel(autor)`

C) O slug único

D) O checklist

B. A tela orienta; o banco decide.

4. Por que mostrar o conteúdo sem HTML cru?

A) Fica mais bonito

B) Um script no texto rodaria no navegador dos leitores

C) Economiza espaço

D) O Supabase exige

B. Conteúdo é texto, nunca código.

○ RECAPITULANDO

A primeira pauta *no papel*

Criamos a tabela artigos com slug único, resumo limitado, tags e rascunho como padrão; regras que deixam só autores criar e cada um editar os seus; e o bucket capas com leitura pública, envio por autores em pastas próprias e limite de tamanho. O editor ganhou Markdown sem HTML cru, pré-visualização, "Salvo às" e publicação com checklist. Bookmark "artigos v1".

artigos + slug

Rascunho privado

tem_papel no INSERT

Bucket capas

Markdown seguro

Checklist de publicação

Próxima aula → [Página de visualização dos artigos](#)

Fontes: docs.lovable.dev (Connect to Supabase: database, storage, migrations) e supabase.com/docs (Row Level Security), consultados em setembro de 2026. As telas são reproduções ilustrativas feitas para o curso, com textos de exemplo.